9

Programação orientada a objetos: Herança

© 2005 Pearson Education do Brasil

OBJETIVOS

- Neste capítulo, você aprenderá:
- Como a herança promove a capacidade de reutilização de software.
- As noções de superclasses e subclasses.
- Como utilizar a palavra-chave `extends` para criar uma classe que herda atributos e comportamentos de outra classe.
- Como utilizar o modificador de acesso `protected` para fornecer acesso de métodos de subclasse a membros de superclasse.
- Como acessar membros de superclasse com `super`.
- Como os construtores são utilizados em hierarquias de herança.
- Os métodos da classe `Object`, a superclasse direta ou indireta de todas as classes em Java.

© 2005 by Pearson Education do Brasil

Sumário

9.1 Introdução

9.2 Superclasses e subclasses

9.3 Membros `protected`

9.4 Relacionamento entre superclasses e subclasses

- 9.4.1 Criando e utilizando uma classe `CommissionEmployee`
- 9.4.2 Criando uma classe `BasePlusCommissionEmployee` sem utilizar herança
- 9.4.3 Criando uma hierarquia de herança `CommissionEmployee–BasePlusCommissionEmployee`
- 9.4.4 Hierarquia de herança `CommissionEmployee–BasePlusCommissionEmployee` com variáveis de instância `protected`
- 9.4.5 Hierarquia de herança `CommissionEmployee–BasePlusCommissionEmployee` com variáveis de instância `private`

© 2005 by Pearson Education do Brasil

Sumário

9.5 Construtores em subclasses

9.6 Engenharia de software com herança

9.7 Classe `Object`

9.8 (Opcional) Estudo de caso de GUIs e imagens gráficas: Exibindo texto e imagens utilizando rótulos

9.9 Conclusão

© 2005 by Pearson Education do Brasil

9.1 Introdução

- **Herança:**
  - **Capacidade de reutilização de software.**
  - **Cria uma nova classe a partir de uma classe existente:**
    - **absorvendo os dados e comportamentos da classe existente; e**
    - **aprimorando-a com novas capacidades.**
  - **A subclasse estende a superclasse.**
    - **Subclasse:**
      - **Grupo mais especializado de objetos.**
      - **Comportamentos herdados da superclasse:**
        - **Podem se personalizar.**
      - **Comportamentos adicionais.**

© 2005 Pearson Education do Brasil

9.1 Introdução (*Continuação*)

- **Hierarquia de classes.**
  - **Superclasse direta:**
    - **Herdada explicitamente (um nível acima na hierarquia).**
  - **Superclasse indireta:**
    - **Herdada de dois ou mais níveis acima na hierarquia.**
  - **Herança única:**
    - **Herda de uma superclasse.**
  - **Herança múltipla:**
    - **Herda de múltiplas superclasses.**
      - **O Java não suporta herança múltipla.**

© 2005 Pearson Education do Brasil

9.2 Superclasses e subclasses

- **Superclasses e subclasses.**
  - **Freqüentemente, um objeto de uma classe também ‘é um’ objeto de uma outra classe.**
    - **Exemplo: Em geometria, um retângulo *é um* quadrilátero.**
      - **A classe `Retângulo` herda da classe `Quadrilátero`.**
      - **`Quadrilátero` : superclasse.**
      - **`Retângulo` : subclasse.**
  - **A superclasse em geral representa um conjunto maior de objetos do que as subclasses.**
    - **Exemplo:**
      - **Superclasse: `Veículo`**
        - **Carros, caminhões, barcos, bicicletas…**
      - **Subclasse: `Carro`**
        - **Subconjunto mais específico e menor de veículos.**

© 2005 Pearson Education do Brasil

Superclasse	Subclasses
Aluno	AlunoDeGraduação, AlunoDePósGraduação
Forma	Círculo, Triângulo, Retângulo
Financiamento	FinanciamentoDeCarro, FinanciamentoDeReformaDaCasa, FinanciamentoDeCasa
Empregado	CorpoDocente, Funcionários
ContaBancária	ContaCorrente, ContaDePoupança

Figura 9.1 | Exemplos de herança.

© 2005 Pearson Education do Brasil

9.2 Superclasses e subclasses (*Continuação*)

- **Hierarquia de herança:**
  - **Relacionamentos de herança: estrutura de hierarquia do tipo árvore.**
  - **Cada classe torna-se:**
    - **Superclasse:**
      - **que fornece membros a outras classes.**
    - **OU**
    - **Subclasse:**
      - **que herda membros de outras classes.**

© 2005 Pearson Education do Brasil

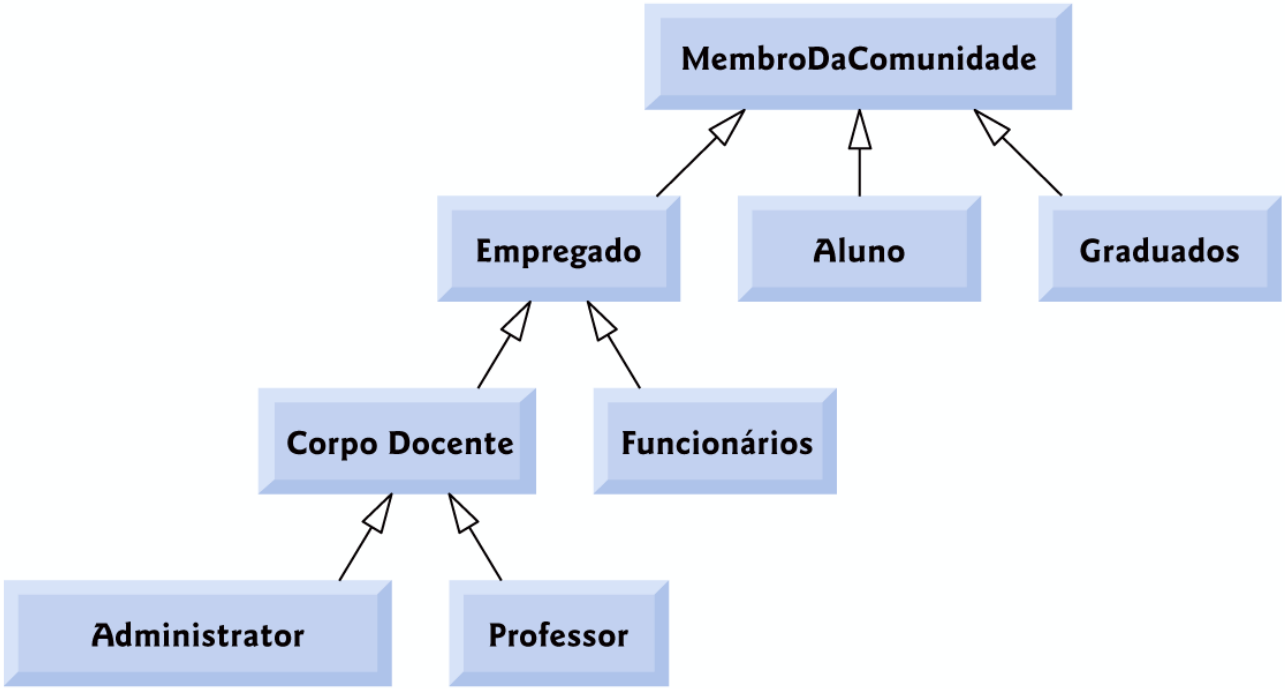

Figura 9.2 | Hierarquia de herança `MembrosDaComunidade` da universidade

© 2005 Pearson Education do Brasil

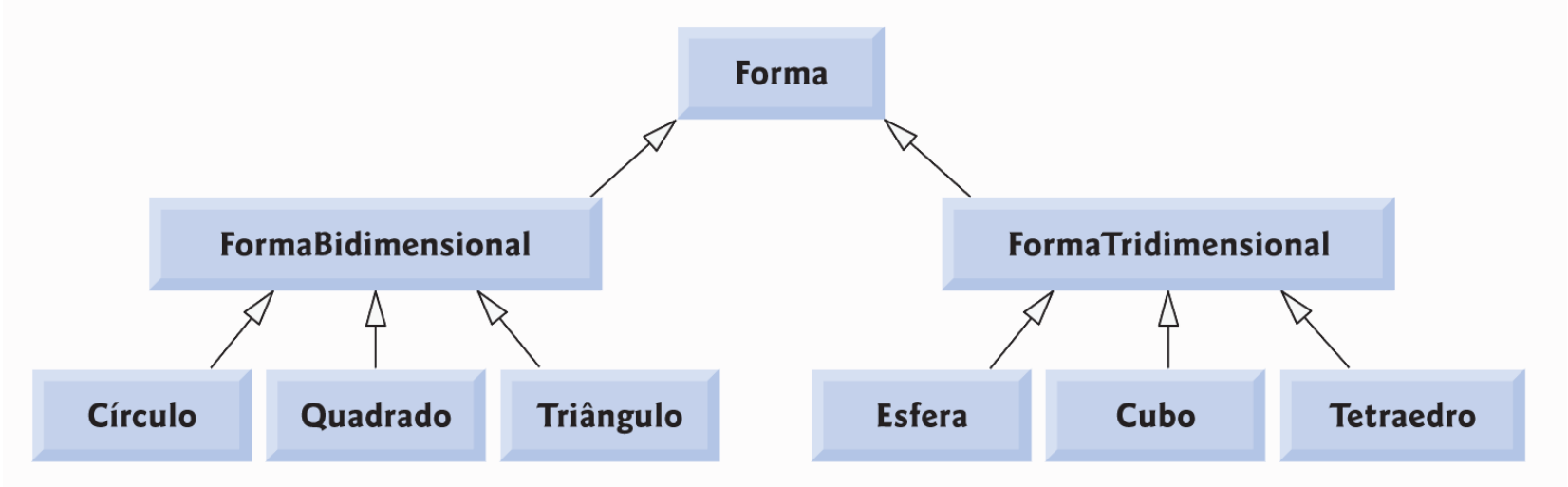

Figura 9.3 | Hierarquia de herança para Forma.

© 2005 Pearson Education do Brasil

9.3 Membros `protected`

- Acesso `protected`:
  - Nível intermediário de proteção entre `public` e `private`.
  - Membros `protected` acessíveis pelos:
    - membros da superclasse;
    - membros da subclasse; e
    - membros da classe no mesmo pacote.
  - Acesso da subclasse ao membro da superclasse:
    - Palavra-chave `super` e um ponto (.)

© 2005 Pearson Education do Brasil

Observação de engenharia de software 9.1

Os métodos de uma subclasse não acessam membros `private` diretamente de sua superclasse. Uma subclasse pode alterar o estado de variáveis de instância `private` da superclasse somente por meio de métodos não-`private` fornecidos na superclasse e herdados pela subclasse.

© 2005 Pearson Education do Brasil

Observação de engenharia de software 9.2

Declarar variáveis de instância `private` ajuda os programadores a testar, depurar e a modificar sistemas corretamente. Se uma subclasse pudesse acessar variáveis de instância `private` da sua superclasse, classes que herdam dessa subclasse também poderiam acessar as variáveis de instância. Isso propagaria acesso ao que devem ser variáveis de instância `private` e os benefícios do ocultamento de informações seriam perdidos.

© 2005 Pearson Education do Brasil

9.4 Relacionamento entre superclasses e subclasses

- **Relacionamento entre superclasse e subclasse.**
  - **Exemplo: Hierarquia de herança `CommissionEmployee/BasePlusCommissionEmployee`**
    - `CommissionEmployee`
      - **Primeiro nome, sobrenome, SSN, taxa de comissão, total de vendas brutas.**
    - `BasePlusCommissionEmployee`
      - **Primeiro nome, sobrenome, SSN, taxa de comissão, total de vendas brutas.**
      - **Salário-base.**

© 2005 Pearson Education do Brasil

9.4.1 Criando e utilizando uma classe `CommissionEmployee`

- Classe `CommissionEmployee`
  - Estende a classe `Object`.
    - Palavra-chave `extends`.
    - Toda classe no Java estende uma classe existente.
      - Exceto `Object`.
    - Toda classe herda os métodos de `Object`.
    - Uma nova classe estende implicitamente `Object`:
      - Se não estender uma outra classe.

© 2005 Pearson Education do Brasil

Observação de engenharia de software 9.3

O compilador Java configura a superclasse de uma classe como `Object` quando a declaração de classe não estender uma superclasse explicitamente.

© 2005 Pearson Education do Brasil

Resumo

```
1  // Fig. 9.4: CommissionEmployee.java
2  // Classe CommissionEmployee representa um empregado com
3
4  public class CommissionEmployee extends Object
5  {
6     private String firstName;
7     private String lastName;
8     private String socialSecurityNumber;
9     private double grossSales; // vendas brutas semanais
10    private double commissionRate; // porcentagem da com
11
12    // construtor de cinco argumentos
13    public CommissionEmployee( String first, String last, String ssn,
14       double sales, double rate )
15    {
16       // chamada implícita para o construtor Object ocorre aqui
17       firstName = first;
18       lastName = last;
19       socialSecurityNumber = ssn;
20       setGrossSales( sales ); // valida e armazena vendas brutas
21       setCommissionRate( rate ); // valida e armazena a taxa de comissão
22    } // fim do construtor CommissionEmployee de cinco argumentos
23
24    // configura o nome
25    public void setFirstName( String first )
26    {
27       firstName = first;
28    } // fim do método setFirstName
29
```

Declara variáveis de instância `private`

A classe `CommissionEmployee` estende a classe `Object`

Chamada implícita ao construtor de `Object`

Inicializa variáveis de instância

Invoca os métodos `setGrossSales` e `setCommissionRate` para validar os dados

nEmployee

.java

de 4)

Linha 4

Linhas 6-10

Linhas 20-21

© 2005 Pearson Education do Brasil

Resumo

```
   // retorna o nome
   public String getFirstName()
   {
      return firstName;
   } // fim do método getFirstName

   // configura o último nome
   public void setLastName( String last )
   {
      lastName = last;
   } // fim do método setLastName

   // retorna o sobrenome
   public String getLastName()
   {
      return lastName;
   } // fim do método getLastName

   // configura o CIC
   public void setSocialSecurityNumber( String ssn )
   {
      socialSecurityNumber = ssn; // deve validar
   } // fim do método setSocialSecurityNumber

   // retorna número do CIC
   public String getSocialSecurityNumber()
   {
      return socialSecurityNumber;
   } // fim do método getSocialSecurityNumber

```

CommissionEmployee.java

(2 de 4)

© 2005 Pearson Education do Brasil

Resumo

```
60    // configura a quantidade de vendas brutas
61    public void setGrossSales( double sales )
62    {
63       grossSales = ( sales < 0.0 ) ? 0.0 : sales;
64    } // fim do método setGrossSales
65
66    // retorna a quantidade de vendas brutas
67    public double getGrossSales()
68    {
69       return grossSales;
70    } // fim do método getGrossSales
71
72    // configura taxa de comissão
73    public void setCommissionRate( double rate )
74    {
75       commissionRate = ( rate > 0.0 && rate < 1.0 ) ? rate : 0.0;
76    } // fim do método setCommissionRate
77
78    // retorna taxa de commissão
79    public double getCommissionRate()
80    {
81       return commissionRate;
82    } // fim do método getCommissionRate
83
84    // calcula os lucros
85    public double earnings()
86    {
87       return commissionRate * grossSales;
88    } // fim do método earnings
89
```

Calcula earnings

CommissionEmployee.java

(3 de 4)

Linhas 85-88

© 2005 Pearson Education do Brasil

Resumo

```
90    // retorna a representação String do objeto CommissionEmployee
91    public String toString()
92    {
93       return String.format( "%s: %s %s\n%s: %s\
94          "commission employee", firstName, last
95          "social security number", socialSecurityNumber,
96          "gross sales", grossSales,
97          "commission rate", commissionRate );
98    } // fim do método toString
99 } // fim da classe CommissionEmployee
```

Sobrescreve o método `toString` da classe `Object`

`CommissionEmployee.java`

(4 de 4)

Linhas 91-98

© 2005 Pearson Education do Brasil

Erro comum de programação 9.1

É um erro de sintaxe sobrescrever um método com um modificador de acesso mais restrito — um método `public` da superclasse não pode se tornar um método `protected` ou `private` na subclasse; um método `protected` da superclasse não pode se tornar um método `private` na subclasse. Fazer isso quebraria o relacionamento ‘é um’ em que se exige que todos os objetos de subclasse sejam capazes de responder a chamadas de método que são feitas para os métodos `public` declarados na superclasse. (C*ontinua*...)

© 2005 Pearson Education do Brasil

Erro comum de programação 9.1 *(Continuação)*

Se um método `public` pudesse ser sobrescrito como um método `protected` ou `private`, os objetos de subclasse não seriam capazes de responder às mesmas chamadas de método como objetos de superclasse. Uma vez que um método é declarado `public` em uma superclasse, o método permanece `public` para todas as subclasses diretas e indiretas dessa classe.

© 2005 Pearson Education do Brasil

Resumo

```
1  // Fig. 9.5: CommissionEmployeeTest.java
2  // Testando a classe CommissionEmployee.
3
4  public class CommissionEmployeeTest
5  {
6     public static void main( String a
7     {
8        // instancia o objeto CommissionEmployee
9        CommissionEmployee employee = new CommissionEmployee(
10          "Sue", "Jones", "222-22-2222", 10000, .06 );
11
12       // obtém os dados de empregado comissionado
13       System.out.println(
14          "Employee information obtained by get methods: \n" );
15       System.out.printf( "%s %s\n", "
16          employee.getFirstName() );
17       System.out.printf( "%s %s\n", "
18          employee.getLastName() );
19       System.out.printf( "%s %s\n", "Social security number is",
20          employee.getSocialSecurityNumber() );
21       System.out.printf( "%s %.2f\n", "Gross sales is",
22          employee.getGrossSales() );
23       System.out.printf( "%s %.2f\n", "Commiss
24          employee.getCommissionRate() );
25
26       employee.setGrossSales( 500 ); // configura vendas brutas
27       employee.setCommissionRate( .1 ); // configura taxa de comissão
28
```

Instancia o objeto `CommissionEmployee`

Utiliza os métodos *get* de `CommissionEmployee` para recuperar os valores da variável de instância do objeto

Utiliza os métodos *set* de `CommissionEmployee` para alterar os valores da variável de instância do objeto

`CommissionEmployeeTest.java`

(1 de 2)

Linhas 9-10

Linhas 15-25

© 2005 Pearson Education do Brasil

Resumo

```
29      System.out.printf( "\n%s:\n\n%s\n",
30         "Updated employee information obtained by toString", employee );
31   } // fim de main
32 } // fim da classe CommissionEmployeeTest
```

Chama implicitamente o método `toString` do objeto

```
Employee information obtained by get methods:

First name is Sue
Last name is Jones
Social security number is 222-22-2222
Gross sales is 10000.00
Commission rate is 0.06

Updated employee information obtained by toString:

commission employee: Sue Jones
social security number: 222-22-2222
gross sales: 500.00
commission rate: 0.10
```

onEmployee
va

(2 de 2)

Linha 30

Saída do programa

© 2005 Pearson Education do Brasil

9.4.2 Criando uma classe `BasePlusCommissionEmployee` sem utilizar herança

- Classe `BasePlusCommissionEmployee`
  - Estende implicitamente `Object`.
  - Boa parte do código é semelhante a `CommissionEmployee`:
    - variáveis de instância `private`;
    - métodos `public`; e
    - construtor.
  - Adições:
    - variável de instância `private` `baseSalary`; e
    - métodos `setBaseSalary` e `getBaseSalary`.

© 2005 Pearson Education do Brasil

Resumo

BasePlusCommissionEmployee.java

```
1  // Fig. 9.6: BasePlusCommissionEmployee.java
2  // A classe BasePlusCommissionEmployee representa um empregado que recebe
3  // um salário-base além da comissão.
4
5  public class BasePlusCommissionEmployee
6  {
7     private String firstName;
8     private String lastName;
9     private String socialSecurityNumber;
10    private double grossSales; // vendas brutas semanais
11    private double commissionRate; // porcentagem da comissão
12    private double baseSalary; // salário-base por semana
13
14    // construtor de seis argumentos
15    public BasePlusCommissionEmployee( String first, String last,
16       String ssn, double sales, double rate, double salary )
17    {
18       // chamada implícita para o construtor Object ocorre aqui
19       firstName = first;
20       lastName = last;
21       socialSecurityNumber = ssn;
22       setGrossSales( sales ); // valida e armazena venda
23       setCommissionRate( rate ); // valida e armazena a taxa de comissão
24       setBaseSalary( salary ); // valida e armazena salário-base
25    } // fim do construtor BasePlusCommissionEmployee de seis argumentos
26
```

Adiciona a variável de instância `baseSalary`

Linha 12

Linha 24

Utiliza o método `setBaseSalary` para validar os dados

© 2005 Pearson Education do Brasil

Resumo

```
   // configura o nome
   public void setFirstName( String first )
   {
      firstName = first;
   } // fim do método setFirstName

   // retorna o nome
   public String getFirstName()
   {
      return firstName;
   } // fim do método getFirstName

   // configura o sobrenome
   public void setLastName( String last )
   {
      lastName = last;
   } // fim do método setLastName

   // retorna o sobrenome
   public String getLastName()
   {
      return lastName;
   } // fim do método getLastName

   // configura o CIC
   public void setSocialSecurityNumber( String ssn )
   {
      socialSecurityNumber = ssn; // deve validar
   } // fim do método setSocialSecurityNumber

```

BasePlusCommissionEmployee.java

(2 de 4)

© 2005 Pearson Education do Brasil

Resumo

BasePlusCommissionEmployee.java

(3 de 4)

```
   // configura o CIC
   public String getSocialSecurityNumber()
   {
      return socialSecurityNumber;
   } // fim do método getSocialSecurityNumber

   // configura quantidade de vendas brutas
   public void setGrossSales( double sales )
   {
      grossSales = ( sales < 0.0 ) ? 0.0 : sales;
   } // fim do método setGrossSales

   // retorna a quantidade de vendas brutas
   public double getGrossSales()
   {
      return grossSales;
   } // fim do método getGrossSales

   // configura a taxa de comissão
   public void setCommissionRate( double rate )
   {
      commissionRate = ( rate > 0.0 && rate < 1.0 ) ? rate : 0.0;
   } // fim do método setCommissionRate

   // retorna taxa de comissão
   public double getCommissionRate()
   {
      return commissionRate;
   } // fim do método getCommissionRate

```

© 2005 Pearson Education do Brasil

Resumo

```
87    // configura salário-base
88    public void setBaseSalary( double salary )
89    {
90       baseSalary = ( salary < 0.0 ) ? 0.0 : salary;
91    } // fim do método setBaseSalary
92
93    // retorna salário-base
94    public double getBaseSalary()
95    {
96       return baseSalary;
97    } // fim do método getBaseSalary
98
99    // calcula lucr
100     public double
101     {
102        return baseSalary + ( commissionRate * grossSales );
103     } // fim do método earnings
104
105     // retorna a representação de String de BasePlusCommissionEmployee
106     public String toString()
107     {
108        return String.format(
109           "%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f\n%s: %.2f",
110           "base-salaried commission employee", firstName, lastName,
111           "social security number", socialSecurityNumber,
112           "gross sales", grossSales, "commission rate", comm
113           "base salary", baseSalary );
114     } // fim do método toString
115  } // fim da classe BasePlusCommissionEmployee
```

O método `setBaseSalary` valida os dados e configura a variável de instância `baseSalary`

O método `getBaseSalary` retorna o valor da variável de instância `baseSalary`

Atualiza o método `earnings` a fim de calcular os rendimentos da comissão-base de um empregado assalariado

Atualiza o método `toString` para exibir o salário-base

`PlusCommission`
`oyee.java`

(4 de 4)

Linhas 88-91

Linhas 94-97

Linha 102

Linhas 108-113

© 2005 Pearson Education do Brasil

Resumo

```
1  // Fig. 9.7: BasePlusCommissionEmployeeTest.java
2  // Testando a classe BasePlusCommissionEmployee.
3
4  public class BasePlusCommissionEmployeeTest
5  {
6     public static void main( String[] args )
7     {
8        // instancia o objeto BasePlusCommissionEmployee
9        BasePlusCommissionEmployee employee =
10          new BasePlusCommissionEmployee(
11          "Bob", "Lewis", "333-33-3333", 5000, .04, 300 );
12
13       // obtém os dados do empregado comissionado com salário-base
14       System.out.println(
15          "Employee information obtained by get methods: \n" );
16       System.out.printf( "%s %s\n",
17          employee.getFirstName() );
18       System.out.printf( "%s %s\n",
19          employee.getLastName() );
20       System.out.printf( "%s %s\n", "Social security number is",
21          employee.getSocialSecurityNumber() );
22       System.out.printf( "%s %.2f\n", "Gross sales is",
23          employee.getGrossSales() );
24       System.out.printf( "%s %.2f\n", "Commission rate is",
25          employee.getCommissionRate() );
26       System.out.printf( "%s %.2f\n", "Base salary is",
27          employee.getBaseSalary() );
28
```

Instancia o objeto `BasePlusCommissionEmployee`

Utiliza os métodos *get* de `BasePluCommissionEmployee` para recuperar os valores da variável de instância do objeto

`sePlusCommission`
`ployeeTest.java`

(1 de 2)

Linha 9-11

Linhas 16-27

© 2005 Pearson Education do Brasil

Resumo

```
29      employee.setBaseSalary( 1000 ); // configura o salário-base
30
31      System.out.printf( "\n%s:\n\n%s\n",
32         "Updated employee information obtained by
33          employee.toString() );
34   } // fim de main
35 } // fim da classe BasePlusCommissionE
```

Utiliza os métodos `setBaseSalary` de `BasePlusCommissionEmployee` para configurar o salário-base

Chama explicitamente o método `toString` do objeto

```
Employee information obtained by get metl

First name is Bob
Last name is Lewis
Social security number is 333-33-3333
Gross sales is 5000.00
Commission rate is 0.04
Base salary is 300.00

Updated employee information obtained by toString:

base-salaried commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000.00
commission rate: 0.04
base salary: 1000.00
```

ssion

`EmployeeTest.java`

(2 de 2)

Linha 29

Linha 33

Saída do programa

© 2005 Pearson Education do Brasil

Observação de engenharia de software 9.4

Copiar e colar código de uma classe para a outra pode espalhar erros por múltiplos arquivos de código-fonte. Para evitar a duplicação de código (e possivelmente erros), utilize herança, em vez da abordagem ‘copiar e colar’, em situações em que você quer que uma classe ‘absorva’ as variáveis de instância e métodos de outra classe.

© 2005 Pearson Education do Brasil

Observação de engenharia de software 9.5

Com a herança, as variáveis de instância comuns e os métodos de todas as classes na hierarquia são declarados em uma superclasse. Quando as alterações são requeridas para esses recursos comuns, os desenvolvedores de software só precisam fazer as alterações na superclasse — as subclasses então herdam as alterações. Sem a herança, as alterações precisariam ser feitas em todos os arquivos de código-fonte que contêm uma cópia do código em questão.

© 2005 Pearson Education do Brasil

9.4.3 Criando uma hierarquia de herança para `CommissionEmployee-BasePlusCommissionEmployee`

- Classe `BasePlusCommissionEmployee2`:
  - estende a classe `CommissionEmployee`;
  - é uma `CommissionEmployee`;
  - tem a variável de instância `baseSalary`;
  - herda membros `public` e `protected`; e
  - construtor não-herdado.

© 2005 Pearson Education do Brasil

Resumo

```
1  // Fig. 9.8: BasePlusCommissionEmployee2.java
2  // BasePlusCommissionEmployee2 herda da classe CommissionEmployee.
3
4  public class BasePlusCommissionEmployee2 extends CommissionEmployee
5  {
6     private double baseSalary; // salário-base por semana
7
8     // construtor de seis argumentos
9     public BasePlusCommissionEmployee2( Stri
10       String ssn, double sales, double rate
11    {
12       // chamada explícita para o construtor CommissionEmployee da superclasse
13       super( first, last, ssn, sales, rate );
14
15       setBaseSalary( amount ); // valida e armazena salário-base
16    } // fim do construtor BasePlusCommiss
17
18    // configura salário-base
19    public void setBaseSalary( double sala
20    {
21       baseSalary = ( salary < 0.0 ) ? 0.0 : salary;
22    } // fim do método setBaseSalary
23
```

BasePlusCommission ee2.java

Classe `BasePlusCommissionEmployee2` é uma subclasse da `CommissionEmployee`

Linha 4

Linha 13

Invoca o construtor da superclasse utilizando a sintaxe de chamada do construtor da superclasse

© 2005 Pearson Education do Brasil

Resumo

BasePlusCommission

(2 de 3)

Linha 34

Linhas 41-46

```
24    // retorna o salário-base
25    public double getBaseSalary()
26    {
27       return baseSalary;
28    } // fim do método getBaseSalary
29
30    // calcula os lucros
31    public double earnings()
32    {
33       // não permitido: commissionRate e grossSales private em superclasse
34       return baseSalary + ( commissionRate * grossSales );
35    } // fim do método earnings
36
37    // retorna represent
38    public String toStri
39    {
40       // não permitido:
41       return String.format(
42          "%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f\n%s: %.2f",
43          "base-salaried commission employee", firstName, lastName,
44          "social security number", socialSecurityNumber,
45          "gross sales", grossSales, "commission rate", commissionRate,
46          "base salary", baseSalary );
47    } // fim do método toString
48 } // fim da classe BasePlusCommissionEmployee2
```

O compilador gera erros porque as variáveis de instância `commissionRate` e `grossSales` da superclasse são `private`

O compilador gera erros porque as variáveis de instância `firstName`, `lastName`, `socialSecurityNumber`, `grossSales` e `commissionRate` da superclasse são `private`

© 2005 Pearson Education do Brasil

Resumo

```
BasePlusCommissionEmployee2.java:34: commissionRate has private access in
CommissionEmployee
      return baseSalary + ( commissionRate * grossSales );
                            ^
BasePlusCommissionEmployee2.java:34: grossSales has private access in
CommissionEmployee
      return baseSalary + ( commissionRate * grossSales );
                                             ^
BasePlusCommissionEmployee2.java:43: firstName has private access in
CommissionEmployee
         "base-salaried commission employee", firstName, lastName,
                                              ^
BasePlusCommissionEmployee2.java:43: lastName has private access in
CommissionEmployee
         "base-salaried commission employee", firstName, lastName,
                                                         ^
BasePlusCommissionEmployee2.java:44: socialSecurityNumber has private access in
CommissionEmployee
         "social security number", socialSecurityNumber,
                                   ^
BasePlusCommissionEmployee2.java:45: grossSales has private access in
CommissionEmployee
         "gross sales", grossSales, "commission rate", commissionRate,
                        ^
BasePlusCommissionEmployee2.java:45: commissionRate has private access in
CommissionEmployee
         "gross sales", grossSales, "commission rate", commissionRate,
                                                       ^
7 errors
```

BasePlusCommissionEmployee2.java

(3 de 3)

O compilador gerou erros

© 2005 Pearson Education do Brasil

Erro comum de programação 9.2

Um erro de compilação ocorre se um construtor de subclasse chamar um de seus construtores de superclasse com argumentos que não correspondem exatamente ao número e tipos de parâmetros especificados em uma das declarações de construtor de superclasse.

© 2005 Pearson Education do Brasil

9.4.4 Hierarquia de herança de `CommissionEmployee-BasePlusCommissionEmployee` com variáveis de instância protected (*Continuação*)

- **Utiliza variáveis de instância `protected`:**
  - **Permite que a classe `BasePlusCommissionEmployee` acesse diretamente as variáveis de instância da superclasse.**
  - **Os membros `protected` da superclasse são herdados por todas as subclasses dessa superclasse.**

© 2005 Pearson Education do Brasil

Resumo

```
1  // Fig. 9.9: CommissionEmployee2.java
2  // Classe CommissionEmployee2 representa um empregado comissionado.
3
4  public class CommissionEmployee2
5  {
6     protected String firstName;
7     protected String lastName;
8     protected String socialSecurityNumber;
9     protected double grossSales; // vendas brutas semanais
10    protected double commissionRate; // porcentagem da comissão
11
12    // construtor de cinco argumentos
13    public CommissionEmployee2( String first, String last, String ssn,
14       double sales, double rate )
15    {
16       // chamada implícita para o construtor Object ocorre aqui
17       firstName = first;
18       lastName = last;
19       socialSecurityNumber = ssn;
20       setGrossSales( sales ); // valida e armazena as vendas brutas
21       setCommissionRate( rate ); // valida e armazena a taxa de comissão
22    } // fim do construtor CommissionEmployee2 de cinco argumentos
23
24    // configura o nome
25    public void setFirstName( String first )
26    {
27       firstName = first;
28    } // fim do método setFirstName
29
```

Declara variáveis de instância `protected`

`CommissionEmployee2.java`

(1 de 4)

Linha 6-10

© 2005 Pearson Education do Brasil

```
   // retorna o nome
   public String getFirstName()
   {
      return firstName;
   } // fim do método getFirstName

   // configura o sobrenome
   public void setLastName( String last )
   {
      lastName = last;
   } // fim do método setLastName

   // retorna o sobrenome
   public String getLastName()
   {
      return lastName;
   } // fim do método getLastName

   // configura o CIC
   public void setSocialSecurityNumber( String ssn )
   {
      socialSecurityNumber = ssn; // deve validar
   } // fim do método setSocialSecurityNumber

   // retorna CIC
   public String getSocialSecurityNumber()
   {
      return socialSecurityNumber;
   } // end method getSocialSecurityNumber

```

Resumo

Commission

`Employee2.java`

(2 de 4)

© 2005 Pearson Education do Brasil

```
   // configura a quantidade de vendas brutas
   public void setGrossSales( double sales )
   {
      grossSales = ( sales < 0.0 ) ? 0.0 : sales;
   } // fim do método setGrossSales

   // retorna a quantidade de vendas brutas
   public double getGrossSales()
   {
      return grossSales;
   } // fim do método getGrossSales

   // configura a taxa de comissão
   public void setCommissionRate( double rate )
   {
      commissionRate = ( rate > 0.0 && rate < 1.0 ) ? rate : 0.0;
   } // fim do método setCommissionRate

   // retorna a taxa de comissão
   public double getCommissionRate()
   {
      return commissionRate;
   } // fim do método getCommissionRate

   // calcula os lucros
   public double earnings()
   {
      return commissionRate * grossSales;
   } // fim do método earnings

```

Resumo

Commission Employee2.java

(3 de 4)

© 2005 Pearson Education do Brasil

```
   // retorna a representação String do objeto CommissionEmployee2
   public String toString()
   {
      return String.format( "%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f",
         "commission employee", firstName, lastName,
         "social security number", socialSecurityNumber,
         "gross sales", grossSales,
         "commission rate", commissionRate );
   } // fim do método toString
} // fim da classe CommissionEmployee2
```

Resumo

Commission Employee2.java

(4 de 4)

© 2005 Pearson Education do Brasil

Resumo

BasePlusCommissionEmployee3.java

```
// Fig. 9.10: BasePlusCommissionEmployee3.java
// BasePlusCommissionEmployee3 herda de CommissionEmployee2 e tem
// acesso a membros protected de CommissionEmployee2.

public class BasePlusCommissionEmployee3 extends CommissionEmployee2
{
   private double baseSalary; // salário-base por semana

   // construtor de seis argumentos
   public BasePlusCommissionEmployee3( String first, String
      String ssn, double sales, double rate, double salary )
   {
      super( first, last, ssn, sales, rate );
      setBaseSalary( salary ); // valida e armazena salário-base
   } // fim do construtor BasePlusCommissionEmployee3 de seis argumentos

   // configura salário-base
   public void setBaseSalary( double salary )
   {
      baseSalary = ( salary < 0.0 ) ? 0.0 : salary;
   } // fim do método setBaseSalary

   // retorna salário-base
   public double getBaseSalary()
   {
      return baseSalary;
   } // fim do método getBaseSalary

```

Deve chamar o construtor da superclasse

Linha 13

© 2005 Pearson Education do Brasil

Resumo

```
29    // calcula os lucros
30    public double earnings()
31    {
32       return baseSalary + ( commissionRate * grossSales );
33    } // fim do método earnings
34
35    // retorna a representação String de BasePlusCommissionEmployee3
36    public String toString()
37    {
38       return String.format(
39          "%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f\n%s: %.2f",
40          "base-salaried commission employee", firstName, lastName,
41          "social security number", socialSecurityNumber,
42          "gross sales", grossSales, "commission rate", commissionRate,
43          "base salary", baseSalary );
44    } // fim do método toString
45 } // fim da classe BasePlusCommissionEmployee3
```

BasePlusCommissionEmployee3.java

Acessa diretamente as variáveis de instância `protected` da superclasse

Linha 32

Linhas 38-43

© 2005 Pearson Education do Brasil

Resumo

```java
// Fig. 9.11: BasePlusCommissionEmployeeTest3.java
// Testando a classe BasePlusCommissionEmployee3.

public class BasePlusCommissionEmployeeTest3
{
   public static void main( String args[] )
   {
      // instancia o objeto BasePlusCommissionEmployee3
      BasePlusCommissionEmployee3 employee =
         new BasePlusCommissionEmployee3(
         "Bob", "Lewis", "333-33-3333", 5000, .04, 300 );

      // obtém os dados do empregado comissionado com salário-base
      System.out.println(
         "Employee information obtained by get methods: \n" );
      System.out.printf( "%s %s\n", "First name is",
         employee.getFirstName() );
      System.out.printf( "%s %s\n", "Last name is",
         employee.getLastName() );
      System.out.printf( "%s %s\n", "Social security number is",
         employee.getSocialSecurityNumber() );
      System.out.printf( "%s %.2f\n", "Gross sales is",
         employee.getGrossSales() );
      System.out.printf( "%s %.2f\n", "Commission rate is",
         employee.getCommissionRate() );
      System.out.printf( "%s %.2f\n", "Base salary is",
         employee.getBaseSalary() );

```

BasePlusCommissionEmployeeTest3.java

(1 de 2)

© 2005 Pearson Education do Brasil

Resumo

```java
      employee.setBaseSalary( 1000 ); // configura o salário-base

      System.out.printf( "\n%s:\n\n%s\n",
         "Updated employee information obtained by toString",
         employee.toString() );
   } // fim de main
} // fim da classe BasePlusCommissionEmployeeTest3
```

```
Employee information obtained by get methods:

First name is Bob
Last name is Lewis
Social security number is 333-33-3333
Gross sales is 5000.00
Commission rate is 0.04
Base salary is 300.00

Updated employee information obtained by toString:

base-salaried commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000.00
commission rate: 0.04
base salary: 1000.00
```

BasePlusCommissionEmployeeTest3.java

(2 de 2)

Saída do programa

© 2005 Pearson Education do Brasil

9.4.4 Hierarquia de herança de `CommissionEmployee-BasePlusCommissionEmployee` utilizando variáveis de instância protegidas (*Continuação*)

- **Utilizando variáveis de instância `protected`.**
  - **Vantagens:**
    - **subclasses podem modificar valores diretamente; e**
    - **pequeno aumento no desempenho.**
      - **Evita overheads da chamada aos métodos *set*/*get*.**
  - **Desvantagens:**
    - **Nenhum teste de validade:**
      - **subclasse pode atribuir valor ilegal.**
    - **Dependente da implementação:**
      - **métodos da subclasse mais provavelmente dependentes da implementação da superclasse; e**
      - **alterações na implementação da superclasse podem resultar em modificações da subclasse.**
        - **Software frágil (quebradiço).**

© 2005 Pearson Education do Brasil

Observação de engenharia de software 9.6

Utilize o modificador de acesso `protected` quando uma superclasse precisar fornecer um método somente para suas subclasses e outras classes no mesmo pacote, mas não para outros clientes.

© 2005 Pearson Education do Brasil

Observação de engenharia de software 9.7

Declarar as variáveis de instância da superclasse `private` (em oposição a `protected`) permite a implementação de superclasse dessas variáveis de instância para alterar sem afetar as implementações de subclasse.

© 2005 Pearson Education do Brasil

Dica de prevenção de erro 9.1

Quando possível, não inclua variáveis de instância `protected` em uma superclasse. Em vez disso, inclua métodos não-`private` que acessam as variáveis de instância `private`. Isso irá assegurar que os objetos da classe mantenham estados consistentes.

© 2005 Pearson Education do Brasil

9.4.5 Hierarquia de herança de `CommissionEmployee–BasePlusCommissionEmployee` com variáveis de instância `private`

- **Reexamine nossa hierarquia de novo:**
  - **Agora utilizando práticas de engenharia de software melhores.**
    - **Declare as variáveis de instância como `private`.**
    - **Forneça os métodos *get* e *set* públicos.**
    - **Utilize o método *get* para obter os valores das variáveis de instância.**

© 2005 Pearson Education do Brasil

Resumo

```
1  // Fig. 9.12: CommissionEmployee3.java
2  // A classe CommissionEmployee3 representa um empregado comissionado.
3
4  public class CommissionEmployee3
5  {
6     private String firstName;
7     private String lastName;
8     private String socialSecurityNumber;
9     private double grossSales; // vendas brutas semanais
10    private double commissionRate; // porcentagem da comissão
11
12    // construtor de cinco argumentos
13    public CommissionEmployee3( String first, String last, String ssn,
14       double sales, double rate )
15    {
16       // chamada implícita para o construtor Object ocorre aqui
17       firstName = first;
18       lastName = last;
19       socialSecurityNumber = ssn;
20       setGrossSales( sales ); // valida e armazena as vendas brutas
21       setCommissionRate( rate ); // valida e armazena a taxa de comissão
22    } // fim do construtor CommissionEmployee3 de cinco argumentos
23
24    // configura o nome
25    public void setFirstName( String first )
26    {
27       firstName = first;
28    } // fim do método setFirstName
29
```

Declara variáveis de instância `private`

`CommissionEmployee3.java`

(1 de 4)

Linhas 6-10

© 2005 Pearson Education do Brasil

```
   // configura o nome
   public String getFirstName()
   {
      return firstName;
   } // fim do método getFirstName

   // configura o sobrenome
   public void setLastName( String last )
   {
      lastName = last;
   } // fim do método setLastName

   // retorna o sobrenome
   public String getLastName()
   {
      return lastName;
   } // fim do método getLastName

   // configura o CIC
   public void setSocialSecurityNumber( String ssn )
   {
      socialSecurityNumber = ssn; // deve validar
   } // fim do método setSocialSecurityNumber

   // retorna o CIC
   public String getSocialSecurityNumber()
   {
      return socialSecurityNumber;
   } // fim do método getSocialSecurityNumber

```

Resumo

Commission Employee3.java

(2 de 4)

© 2005 Pearson Education do Brasil

```
   // configura a quantidade de vendas brutas
   public void setGrossSales( double sales )
   {
      grossSales = ( sales < 0.0 ) ? 0.0 : sales;
   } // fim do método setGrossSales

   // retorna a quantidade de vendas brutas
   public double getGrossSales()
   {
      return grossSales;
   } // fim do método getGrossSales

   // configura a taxa de comissão
   public void setCommissionRate( double rate )
   {
      commissionRate = ( rate > 0.0 && rate < 1.0 ) ? rate : 0.0;
   } // fim do método setCommissionRate

   // retorna a taxa de comissão
   public double getCommissionRate()
   {
      return commissionRate;
   } // fim do método getCommissionRate

```

Resumo

Commission Employee3.java

(3 de 4)

© 2005 Pearson Education do Brasil

Resumo

```
84    // calcula os lucros
85    public double earnings()
86    {
87       return getCommissionRate() * getGrossSales();
88    } // fim do método earnings
89
90    // retorna a representação String do objeto CommissionEmpl
91    public String toString()
92    {
93       return String.format( "%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f",
94          "commission employee", getFirstName(), getLastName(),
95          "social security number", getSocialSecurityNumber(),
96          "gross sales", getGrossSales(),
97          "commission rate", getCommissionRate() );
98    } // fim do método toString
99 } // fim da classe CommissionEmployee3
```

Utiliza os métodos *get* para obter os valores das variáveis de instância

Employee3.java

(4 de 4)

Linha 87

Linhas 94-97

© 2005 Pearson Education do Brasil

Resumo

```java
// Fig. 9.13: BasePlusCommissionEmployee4.java
// Classe BasePlusCommissionEmployee4 herda de CommissionEmployee3 e
// acessa os dados privados de CommissionEmployee3 via os métodos
// CommissionEmployee3 e public.

public class BasePlusCommissionEmployee4 extends CommissionEmployee3
{
   private double baseSalary; // salário-base por semana

   // construtor de seis argumentos
   public BasePlusCommissionEmployee4( String first, String last,
      String ssn, double sales, double rate, double salary )
   {
      super( first, last, ssn, sales, rate );
      setBaseSalary( salary ); // valida e armazena salário-base
   } // fim do construtor BasePlusCommissionEmployee4 de seis argumentos

   // configura o salário-base
   public void setBaseSalary( double salary )
   {
      baseSalary = ( salary < 0.0 ) ? 0.0 : salary;
   } // fim do método setBaseSalary

```

Herda de `CommissionEmployee3`

BasePlusCommissionEmployee4.java

(1 de 2)

© 2005 Pearson Education do Brasil

Resumo

```
24    // retorna o salário-base
25    public double getBaseSalary()
26    {
27       return baseSalary;
28    } // fim do método getBaseSalary
29
30    // calcula os lucros
31    public double earnings()
32    {
33       return getBaseSalary() + super.earnings();
34    } // fim do método earnings
35
36    // retorna a representação String de BasePlusCommissionEmplo
37    public String toString()
38    {
39       return String.format( "%s %s\n%s: %.2f", "base-salaried",
40          super.toString(), "base salary", getBaseSalary() );
41    } // fim do método toString
42 } // fim da classe BasePlusCommissionEmployee4
```

Invoca um método sobrescrito da superclasse a partir de uma subclasse

Utiliza os métodos *get* para obter os valores das variáveis de instância

Linhas 40

Invoca um método sobrescrito da superclasse a partir de uma subclasse

© 2005 Pearson Education do Brasil

Erro comum de programação 9.3

Quando um método da superclasse é sobrescrito em uma subclasse, a versão da subclasse freqüentemente chama a versão da superclasse para realizar uma parte do trabalho. A falha em prefixar o nome do método da superclasse com a palavra-chave `super` e um ponto (.) separador ao referenciar o método da superclasse faz com que o método da subclasse chame a si mesmo, criando um erro chamado recursão infinita. A recursão, utilizada corretamente, é uma capacidade poderosa discutida no Capítulo 15, Recursão.

© 2005 Pearson Education do Brasil

Resumo

```
1  // Fig. 9.14: BasePlusCommissionEmployeeTest4.java
2  // Testando a classe BasePlusCommissionEmployee4.
3
4  public class BasePlusCommissionEmployeeTest4
5  {
6     public static void main( String args[] )
7     {
8        // instancia o objeto BasePlusCommissionEmployee4
9        BasePlusCommissionEmployee4 employee =
10          new BasePlusCommissionEmployee4(
11          "Bob", "Lewis", "333-33-3333", 5000, .04, 300 );
12
13       // obtém os dados do empregado comissionado com salário-base
14       System.out.println(
15          "Employee information obtained by get methods: \n" );
16       System.out.printf( "%s %s\n", "First name is",
17          employee.getFirstName() );
18       System.out.printf( "%s %s\n", "Last name is",
19          employee.getLastName() );
20       System.out.printf( "%s %s\n", "Social security number is",
21          employee.getSocialSecurityNumber() );
22       System.out.printf( "%s %.2f\n", "Gross sales is",
23          employee.getGrossSales() );
24       System.out.printf( "%s %.2f\n", "Commission rate is",
25          employee.getCommissionRate() );
26       System.out.printf( "%s %.2f\n", "Base salary
27          employee.getBaseSalary() );
28
```

Cria um objeto `BasePlusCommissionEmployee4`.

(1 de 2)

Linhas 9-11

Linhas 16-25

Utiliza os métodos *get* herdados para acessar as variáveis de instância `private` herdadas

Utiliza o método *get* de `BasePlusCommissionEmployee4` para acessar a variável de instância privada.

© 2005 Pearson Education do Brasil

Resumo

```
29      employee.setBaseSalary( 1000 ); // configura o salário-base
30
31      System.out.printf( "\n%s:\n\n%s\n",
32         "Updated employee information obtaine
33         employee.toString() );
34   } // fim de main
35 } // fim da classe BasePlusCommissionEmployeeT
```

Utiliza o método *set* de `BasePlusCommissionEmployee4` para modificar a variável de instância `private baseSalary`.

```
Employee information obtained by get methods:

First name is Bob
Last name is Lewis
Social security number is 333-33-3333
Gross sales is 5000.00
Commission rate is 0.04
Base salary is 300.00

Updated employee information obtained by toString:

base-salaried commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000.00
commission rate: 0.04
base salary: 1000.00
```

(2 de 2)

© 2005 Pearson Education do Brasil

9.5 Construtores em subclasses

- **Instanciando objetos de subclasse.**
  - **Cadeia de chamadas ao construtor:**
    - **O construtor da subclasse invoca o construtor da superclasse implícita ou explicitamente.**
    - **Base da hierarquia de herança:**
      - **O último construtor chamado na cadeia é o construtor de `Object`.**
      - **O corpo do construtor de subclasse original termina a execução por último.**
      - **Exemplo: Hierarquia de `CommissionEmployee3-BasePlusCommissionEmployee4`**
        - **O construtor de `CommissionEmployee3` chamado em penúltimo lugar (o último é o construtor de `Object`).**
        - **O corpo do construtor de `CommissionEmployee3` termina a execução em segundo lugar (o primeiro é o corpo do construtor de `Object`).**

© 2005 Pearson Education do Brasil

Observação de engenharia de software 9.8

Quando um programa cria um objeto de subclasse, o construtor de subclasse imediatamente chama o construtor de superclasse (explicitamente, via `super` ou implicitamente). O corpo do construtor de superclasse executa para inicializar as variáveis de instância da superclasse que fazem parte do objeto de subclasse, então o corpo do construtor de subclasse executa para inicializar variáveis de instância somente de subclasse. (C*ontinua*…)

© 2005 Pearson Education do Brasil

Observação de engenharia de software 9.8 (*Continuação)*

O Java assegura que mesmo que um construtor não atribua um valor a uma variável de instância, a variável ainda é inicializada como seu valor-padrão (por exemplo, `0` para tipos numéricos primitivos, `false` para `boolean`s, `null` para referências).

© 2005 Pearson Education do Brasil

Resumo

```
1  // Fig. 9.15: CommissionEmployee4.java
2  // Classe CommissionEmployee4 representa um empregado comissionado.
3
4  public class CommissionEmployee4
5  {
6     private String firstName;
7     private String lastName;
8     private String socialSecurityNumber;
9     private double grossSales; // vendas brutas semanais
10    private double commissionRate; // porcentagem da comissão
11
12    // construtor de argumentos
13    public CommissionEmployee4( String first, String last, String ssn,
14       double sales, double rate )
15    {
16       // chamada implícita para o construtor Object ocorre aqui
17       firstName = first;
18       lastName = last;
19       socialSecurityNumber = ssn;
20       setGrossSales( sales ); // valida e ar
21       setCommissionRate( rate ); // valida e
22
23       System.out.printf(
24          "\nCommissionEmployee4 constructor:\n%s\n", this );
25    } // fim do construtor CommissionEmployee4 de cinco argumentos
26
```

O construtor gera uma mensagem para demonstrar a ordem de chamada de método.

CommissionEmployee4.java

(1 de 4)

Linhas 23-24

© 2005 Pearson Education do Brasil

Resumo

```
   // configura o nome
   public void setFirstName( String first )
   {
      firstName = first;
   } // fim do método setFirstName

   // retorna o nome
   public String getFirstName()
   {
      return firstName;
   } // fim do método getFirstName

   // configura o sobrenome
   public void setLastName( String last )
   {
      lastName = last;
   } // fim do método setLastName

   // retorna o sobrenome
   public String getLastName()
   {
      return lastName;
   } // fim do método getLastName

   // configura o CIC
   public void setSocialSecurityNumber( String ssn )
   {
      socialSecurityNumber = ssn; // deve validar
   } // fim do método setSocialSecurityNumber

```

CommissionEmployee4.java

(2 de 4)

© 2005 Pearson Education do Brasil

Resumo

```
   // retorna o CIC
   public String getSocialSecurityNumber()
   {
      return socialSecurityNumber;
   } // fim do método getSocialSecurityNumber

   // configura a quantidade de vendas brutas
   public void setGrossSales( double sales )
   {
      grossSales = ( sales < 0.0 ) ? 0.0 : sales;
   } // fim do método setGrossSales

   // retorna a quantidade de vendas brutas
   public double getGrossSales()
   {
      return grossSales;
   } // fim do método getGrossSales

   // configura a taxa de comissão
   public void setCommissionRate( double rate )
   {
      commissionRate = ( rate > 0.0 && rate < 1.0 ) ? rate : 0.0;
   } // fim do método setCommissionRate

```

CommissionEmployee4.java

(3 de 4)

© 2005 Pearson Education do Brasil

Resumo

```
   // retorna a taxa de comissão
   public double getCommissionRate()
   {
      return commissionRate;
   } // fim do método getCommissionRate

   // calcula os lucros
   public double earnings()
   {
      return getCommissionRate() * getGrossSales();
   } // fim do método earnings

   // retorna a representação String de objeto CommissionEmployee4
   public String toString()
   {
      return String.format( "%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f",
         "commission employee", getFirstName(), getLastName(),
         "social security number", getSocialSecurityNumber(),
         "gross sales", getGrossSales(),
         "commission rate", getCommissionRate() );
    } // fim do método toString
 } // fim da classe CommissionEmployee4
```

CommissionEmployee4.java

(4 de 4)

© 2005 Pearson Education do Brasil

Resumo

```
1  // Fig. 9.16: BasePlusCommissionEmployee5.java
2  // Declaração de classe BasePlusCommissionEmployee5.
3
4  public class BasePlusCommissionEmployee5 extends CommissionEmployee4
5  {
6     private double baseSalary; // salário-base por semana
7
8     // construtor de seis argumentos
9     public BasePlusCommissionEmployee5( String first, String last,
10       String ssn, double sales, double rate, double salary )
11    {
12       super( first, last, ssn, sales, rate );
13       setBaseSalary( salary ); // valida e ar
14
15       System.out.printf(
16          "\nBasePlusCommissionEmployee5 constructor:\n%s\n", this );
17    } // fim do construtor BasePlusCommissionEmployee5 de seis argumentos
18
19    // configura salário-base
20    public void setBaseSalary( double salary )
21    {
22       baseSalary = ( salary < 0.0 ) ? 0.0 : salary;
23    } // fim do método setBaseSalary
24
```

O construtor gera uma mensagem para demonstrar a ordem de chamada de método.

BasePlusCommissionEmployee5.java

(1 de 2)

Linhas 15-16

© 2005 Pearson Education do Brasil

Resumo

```
   // retorna salário-base
   public double getBaseSalary()
   {
      return baseSalary;
   } // fim do método getBaseSalary

   // calcula os lucros
   public double earnings()
   {
      return getBaseSalary() + super.earnings();
   } // fim do método earnings

   // retorna a representação String de BasePlusCommissionEmployee5
   public String toString()
   {
      return String.format( "%s %s\n%s: %.2f", "base-salaried",
         super.toString(), "base salary", getBaseSalary() );
   } // fim do método toString
} // fim da classe BasePlusCommissionEmployee5
```

BasePlusCommissionEmployee5.java

(2 de 2)

© 2005 Pearson Education do Brasil

Resumo

```
// Fig. 9.17: ConstructorTest.java
// Exibe ordem em que construtores de superclasse e subclasse são chamados.

public class ConstructorTest
{
   public static void main( String args[] )
   {
      CommissionEmployee4 employee1 = new CommissionEmployee4(
         "Bob", "Lewis", "333-33-3333", 5000, .04 );

      System.out.println();
      BasePlusCommissionEmployee5 employee2 =
         new BasePlusCommissionEmployee5(
         "Lisa", "Jones", "555-55-5555", 2000, .06, 800 );

      System.out.println();
      BasePlusCommissionEmployee5 employee3 =
         new BasePlusCommissionEmployee5(
         "Mark", "Sands", "888-88-8888", 8000, .15, 2000 );
   } // fim de main
} // fim da classe ConstructorTest
```

Instancia o objeto `CommissionEmployee4`

Instancia dois objetos `BasePlusCommissionEmployee5` para demonstrar a ordem das chamadas de método do construtor da subclasse e da superclasse.

ConstructorTest.java

(1 de 2)

© 2005 Pearson Education do Brasil

Resumo

```
CommissionEmployee4 constructor:
commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000.00
commission rate: 0.04

CommissionEmployee4 constructor:
base-salaried commission employee: Lisa Jones
social security number: 555-55-5555
gross sales: 2000.00
commission rate: 0.06
base salary: 0.00

BasePlusCommissionEmployee5 constructor:
base-salaried commission employee: Lisa Jones
social security number: 555-55-5555
gross sales: 2000.00
commission rate: 0.06
base salary: 800.00


CommissionEmployee4 constructor:
base-salaried commission employee: Mark Sands
social security number: 888-88-8888
gross sales: 8000.00
commission rate: 0.15
base salary: 0.00

BasePlusCommissionEmployee5 constructor:
base-salaried commission employee: Mark Sands
social security number: 888-88-8888
gross sales: 8000.00
commission rate: 0.15
base salary: 2000.00
```

O corpo do construtor da subclasse `BasePlusCommissionEmployee5` é executado depois que a execução do construtor da superclasse `CommissionEmployee4` termina.

`ConstructorTest.java`

(2 de 2)

© 2005 Pearson Education do Brasil

9.6 Engenharia de software com herança

- **Personalizando software existente:**
  - **Herda de classes existentes:**
    - **Inclui membros adicionais.**
    - **Redefine os membros da superclasse.**
    - **Nenhum acesso direto ao código-fonte da superclasse.**
      - **Vincula ao código-objeto.**
  - **Fornecedores de software independentes (*independent software vendors* – ISVs):**
    - **Desenvolve código proprietário para venda/licenciamento.**
      - **Disponível no formato de código-objeto.**
    - **Usuários derivam novas classes.**
      - **Sem acessar o código-fonte proprietário do ISV.**

© 2005 Pearson Education do Brasil

Observação de engenharia de software 9.9

Apesar do fato de que herdar de uma classe não requer acesso ao código-fonte da classe, freqüentemente, os desenvolvedores insistem em examinar o código-fonte para entender como a classe é implementada. Os desenvolvedores na indústria querem assegurar que eles estão estendendo uma classe sólida — por exemplo, uma classe que executa bem e é implementada seguramente.

© 2005 Pearson Education do Brasil

Observação de engenharia de software 9.10

Na etapa de projeto em um sistema orientado a objetos, o projetista freqüentemente descobre que certas classes estão intimamente relacionadas. O projetista deve ‘fatorar’ as variáveis de instância e métodos comuns e colocá-los em uma superclasse. Então o projetista deve utilizar a herança para desenvolver subclasses, especializando-as com capacidades além daquelas herdadas da superclasse.

© 2005 Pearson Education do Brasil

Observação de engenharia de software 9.11

Declarar uma subclasse não afeta o código-fonte da sua superclasse. A herança preserva a integridade da superclasse.

© 2005 Pearson Education do Brasil

Observação de engenharia de software 9.12

Assim como os projetistas de sistemas não-orientados a objetos devem evitar a proliferação de métodos, os projetistas de sistemas orientados a objetos devem evitar a proliferação de classes. Essa proliferação cria problemas de gerenciamento e pode prejudicar a capacidade de reutilização de software, porque em uma enorme biblioteca de classes torna-se difícil para um cliente localizar as classes mais apropriadas. A alternativa é criar menos classes que fornecem funcionalidades mais substanciais, mas isso pode se tornar complicado.

© 2005 Pearson Education do Brasil

Dica de desempenho 9.1

Se as subclasses são maiores do que precisam ser (isto é, se elas contêm funcionalidades demais), recursos de memória e processamento podem ser desperdiçados. Estenda a superclasse que contém as funcionalidades mais próximas das funcionalidades que precisam ser criadas.

© 2005 Pearson Education do Brasil

9.7 A classe `Object`

- **Métodos da classe `Object`:**
  - `clone`
  - `equals`
  - `finalize`
  - `getClass`
  - `hashCode`
  - `notify`, `notifyAll`, `wait`
  - `toString`

© 2005 Pearson Education do Brasil

Método	Descrição
`Clone`	Esse método **`protected`**, que não aceita nenhum argumento e retorna uma referência **`Object`**, faz uma cópia do objeto em que é chamado. Quando a clonagem for necessária para os objetos de uma classe, a classe deve sobrescrever o método **`clone`** como um método **`public`** e deve implementar a interface **`Cloneable`** (pacote **`java.lang`**). A implementação padrão desse método realiza a chamada **cópia superficial** — os valores da variável de instância em um objeto são copiados em outro objeto do mesmo tipo. Para tipos por referência, apenas as referências são copiadas. Uma típica implementação do método **`clone`** sobrescrito realizaria uma **cópia em profundidade** que cria um novo objeto para cada variável de instância de tipo por referência. Há muitas sutilezas para sobrescrever método **`clone`**. Você pode aprender mais sobre a clonagem no seguinte artigo: **`java.sun.com/developer/JDCTechTips/2001/tt0306.html`**

Figura 9.18 | Métodos `Object` que são herdados direta ou indiretamente por todas as classes. (Parte 1 de 4.)

© 2005 Pearson Education do Brasil

Método	Descrição
`Equals`	Esse método compara dois objetos quanto à igualdade e retorna `true` se eles forem iguais, caso contrário, retorna `false`. O método aceita qualquer `Object` como um argumento. Quando os objetos de uma classe particular precisarem ser comparados quanto à igualdade, a classe deve sobrescrever o método `equals` para comparar o conteúdo dos dois objetos. A implementação do método deve atender aos seguintes requisitos: • Você deve retornar `false` se o argumento for `null`. • Você deve retornar `true` se um objeto for comparado com ele mesmo, como em `object1.equals( object1 )`. • Você só deve retornar `true` se tanto `object1.equals( object2 )` como `object2.equals( object1 )` retornarem `true`. • Para três objetos, se `object1.equals( object2 )` retornar `true` e `object2.equals( object3 )` retornar `true`, então `object1.equals( object3 )` também deve retornar `true`. • Se `equals` for chamado múltiplas vezes com os dois objetos e os objetos não mudarem, o método deve retornar `true` consistentemente se os objetos forem iguais e, `false`, caso contrário. Uma classe que sobrescreve `equals` também deve sobrescrever `hashCode` para assegurar que objetos iguais tenham códigos de hash idênticos. A implementação `equals` padrão utiliza o operador == para determinar se duas referências *referenciam o mesmo objeto* na memória. A Seção 29.3.3 demonstra o método `equals` da classe `String` e diferencia entre comparar objetos `String` com == e com `equals`.

Figura. 9.18 | Métodos `Object` que são herdados direta ou indiretamente por todas as classes. (Parte 2 de 4.)

© 2005 Pearson Education do Brasil

Método	Descrição
`finalize`	Esse método `protected` (introduzido na Seção 8.10 e Seção 8.11) é chamado pelo coletor de lixo para realizar a limpeza de término em um objeto antes de o coletor de lixo reivindicar a memória do objeto. Não é garantido que o coletor de lixo irá reivindicar um objeto, então não é possível garantir que o método `finalize` do objeto executará. O método deve especificar uma lista vazia de parâmetro e deve retornar `void`. A implementação padrão desse método serve como um marcador de lugar que não faz nada.
`getClass`	Todo objeto no Java conhece seu próprio tipo em tempo de execução. O método `getClass` (utilizado na Seção 10.5 e Seção 21.3) retorna um objeto de classe `Class` (pacote `java.lang`) que contém as informações sobre o tipo de objeto, como seu nome de classe (retornado pelo método `Class getName`). Você pode aprender mais sobre a classe `Class` na documentação de API on-line em `java.sun.com/j2se/5.0/docs/api/java/lang/Class.html`.

Figura 9.18 | Os métodos `Object` que são herdados direta ou indiretamente por todas as classes. (Parte 3 de 4.)

© 2005 Pearson Education do Brasil

Método	Descrição
`hashCode`	Uma tabela de hash é uma estrutura de dados (discutida na Seção 19.10) que relaciona um objeto, chamado chave, com outro objeto, chamado valor. Ao inserir inicialmente um valor em uma tabela de hash, o método `hashCode` da chave é chamado. O valor do código de hash retornado é utilizado pela tabela de hash para determinar em qual localização inserir o valor correspondente. O código de hash da chave também é utilizado pela tabela de hash para localizar o valor correspondente da chave.
`notify`, `notifyAll`, `wait`	Os métodos `notify`, `notifyAll` e as três versões sobrecarregadas de `wait` são relacionados à multithreading, que é discutido no Capítulo 23. No J2SE 5.0, o modelo multithreading mudou substancialmente, mas esses recursos continuam a ser suportados.
`toString`	Esse método (introduzido na Seção 9.4.1) retorna uma representação `String` de um objeto. A implementação-padrão desse método retorna o nome de pacote e o nome de classe da classe do objeto seguido por uma representação hexadecimal do valor retornado pelo método `hashCode` do objeto.

Figura 9.18 | Métodos `Object` que são herdados direta ou indiretamente por todas as classes. (Parte 4 de 4.)

© 2005 Pearson Education do Brasil